

CloudPack: A Cloud-Native, Microservices-Based DevOps Deployment of an Agentic AI Packing Assistant

Submitted by:

Drishtita

Registration No:

12303458

Programme and Section:

BTECH CSE K3O2344

Course Code: INT378

Under the Guidance of
Mr. Ajay Kumar Badhan

Discipline of CSE/IT



L OVELY
P ROFESSIONAL
U NIVERSITY

Transforming Education Transforming India

1. Introduction & Background

The Smart Packing List Agent is an existing agentic artificial intelligence system, originally developed as part of a separate Agentic AI course project. The agent follows a plan-act loop: it accepts a natural-language trip goal from the user (destination, duration, and trip type), invokes a weather tool that retrieves real forecast data from the OpenWeatherMap API, and invokes an item tool that adds packing items to a running list, each justified by a reason tied to the forecast or the nature of the trip. The agent maintains session memory of the active trip so that later turns can build on earlier decisions without re-deriving them from scratch, and all packing decisions are made by a large language model accessed through Groq's OpenAI-compatible API, with no hardcoded packing rules present in the codebase.

While the underlying agent logic is functionally complete, it currently exists as a single monolithic notebook-style application with no deployment infrastructure, no automation, and no operational visibility. The CloudPack project proposes to take this existing agent as a fixed starting point and re-architect, containerize, orchestrate, provision, automate, and monitor it using an industry-representative DevOps toolchain. The remainder of this synopsis is concerned almost entirely with this DevOps transformation, which constitutes the actual technical and evaluative focus of the project.

2. Problem Statement / Motivation

A monolithic, notebook-based implementation of an agentic system is unsuitable for any real deployment context for several concrete reasons. First, it cannot be scaled selectively: if the weather-fetching logic becomes a bottleneck, the entire application must be scaled as one unit, wasting compute resources on components that are not under load. Second, it has no resilience: a single unhandled exception can terminate the entire process, and there is no mechanism to detect or recover from such failures automatically. Third, all configuration and credentials (API keys for Groq and OpenWeatherMap) are typically embedded or loaded ad hoc, with no standardized, auditable mechanism for secret management. Fourth, there is no repeatable or version-controlled way to provision the infrastructure the application runs on, making environment drift and manual configuration errors likely. Fifth, there is no automated pipeline connecting a code change to a live, tested deployment, so releases depend on manual, error-prone steps. Finally, there is no visibility into the running system's behaviour in production — no metrics on latency, error rates, or throughput are collected or visualized.

DevOps practices directly address each of these deficiencies: containerization enables independent packaging and scaling; container orchestration provides self-healing and service discovery; Infrastructure as Code provides reproducible and version-controlled environments; a CI/CD pipeline provides automated, tested, and repeatable delivery from commit to production; and a monitoring stack provides continuous observability of the live system. The CloudPack project is motivated by the need to demonstrate all of these practices applied coherently to a single, realistic application.

3. Objectives

- To decompose the monolithic agent into independently deployable microservices, each with a clearly bounded responsibility.
- To containerize each microservice using Docker, producing one image per service with an appropriately layered Dockerfile.
- To orchestrate the containerized microservices using Kubernetes, with Deployments, Services, ConfigMaps, and Secrets for each component.
- To provision all underlying AWS infrastructure declaratively using Terraform, eliminating manual console-based configuration.

- To deploy the orchestrated system onto AWS using managed or self-managed Kubernetes and a container image registry.
- To build a Jenkins-based CI/CD pipeline that automatically tests, builds, pushes, and deploys the system on every code change.
- To instrument every microservice for observability using Prometheus for metrics collection and Grafana for dashboarding.

4. Proposed System Architecture

The target architecture decomposes the original monolithic agent into four independently containerized and independently deployable microservices, communicating over REST:

- agent-service — orchestrates the plan-act loop, invokes the LLM via the Groq API, and coordinates calls to the other services.
- weather-service — wraps the OpenWeatherMap API call in isolation, allowing it to be scaled or cached independently of the reasoning logic.
- memory-service — manages session and trip state, backed by a persistent store (Redis or PostgreSQL) rather than an in-process object, since Kubernetes pods are ephemeral and state must survive pod restarts.
- api-gateway — a single external entry point that routes incoming requests to the appropriate internal service, following the standard API gateway pattern used in production microservice systems.

Each service is packaged as its own container image, deployed and scaled independently, and reachable by the other services through internal networking rather than direct in-process function calls.

Diagram 1: Architecture Diagram / Agent Workflow (Data Flow Diagram)

The diagram below traces the agent's own request-handling workflow end-to-end: the user's trip request enters through api-gateway, is routed to agent-service, and agent-service in turn drives the plan-act loop by calling weather-service (which itself calls the external OpenWeatherMap API), calling the Groq LLM for reasoning, and reading/writing session state through memory-service (backed by the persistent Redis / PostgreSQL store), before the final packing list is returned to the user through api-gateway.

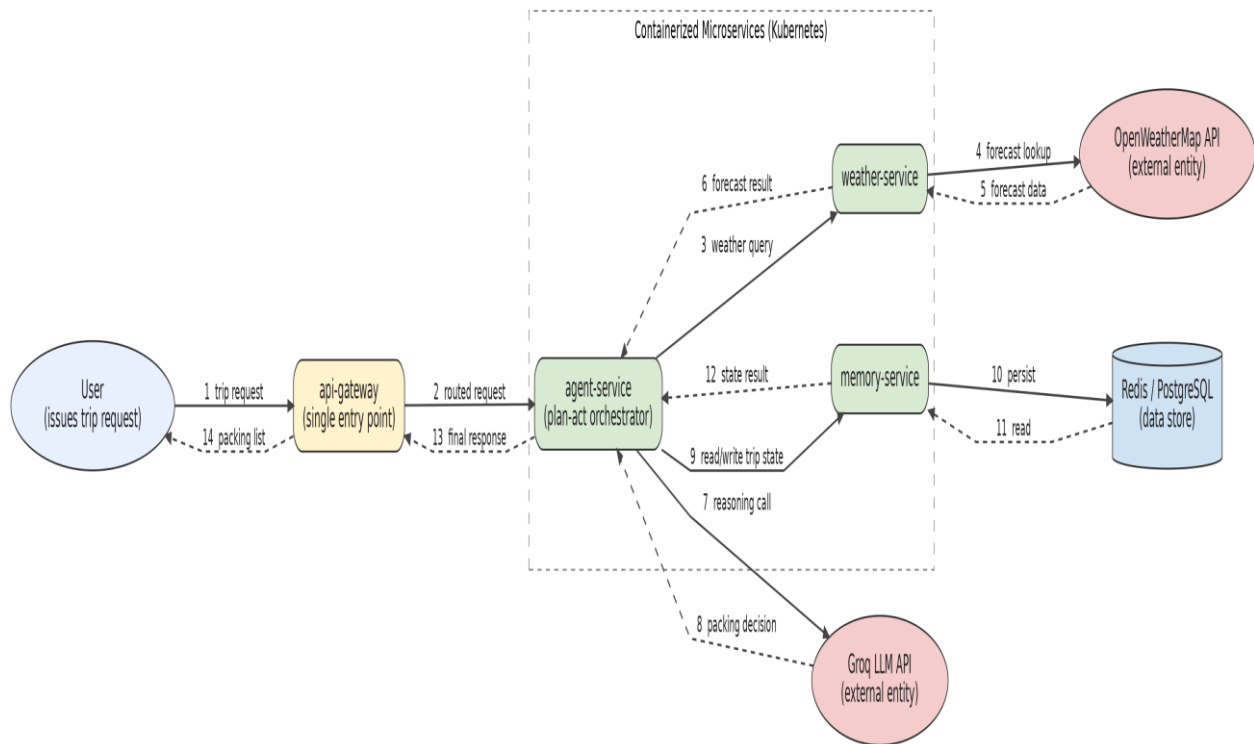


Figure 1. Agent workflow / architecture data-flow diagram — numbered arrows show data flow between the user, api-gateway, the microservices, the external APIs, and the persistent store.

5. DevOps Toolchain and Methodology

5.1 Containerization (Docker)

Each microservice — agent-service, weather-service, memory-service, and api-gateway — is packaged into its own Docker image, built from its own Dockerfile. Each Dockerfile follows a layered structure: a minimal base image, a dependency-installation layer (cached separately so that dependency changes do not force a full rebuild of application code), and an application-code layer copied in last. Isolating each service into its own image allows each to be scaled, updated, rolled back, and redeployed independently of the others, and ensures that a change to one service's dependencies cannot break or unnecessarily rebuild an unrelated service.

5.2 Container Orchestration (Kubernetes)

Each of the four containerized microservices is deployed to Kubernetes as its own Deployment object, fronted by its own Service object for stable internal networking. Environment variables and non-sensitive configuration are supplied via ConfigMaps, while sensitive values — the Groq API key, the OpenWeatherMap API key, and database credentials — are supplied via Kubernetes Secrets rather than being embedded in images or manifests. Kubernetes provides horizontal scaling of any individual service under load, self-healing by automatically restarting or rescheduling failed pods, and service discovery, whereby, for example, agent-service reaches memory-service simply by addressing its Kubernetes Service DNS name rather than a hardcoded IP address.

5.3 Infrastructure as Code (Terraform)

All underlying AWS infrastructure required to run the cluster — the VPC and subnets, the EKS cluster or equivalent compute layer, ECR repositories for each service's container images, and the IAM roles and policies governing access between components — is provisioned declaratively using Terraform rather than manually through the AWS console. This makes the infrastructure reproducible, version-controlled alongside the application code, and auditable, since any change to the infrastructure is expressed as a reviewable diff to Terraform configuration files rather than an untracked manual action.

5.4 Cloud Deployment (AWS)

The system is deployed on Amazon Web Services. Amazon ECR (Elastic Container Registry) stores the versioned Docker images built for each microservice. Amazon EKS (Elastic Kubernetes Service), or a self-managed Kubernetes cluster on EC2 where EKS is unavailable, hosts the running cluster and all Deployments and Services described in Section 5.2. Where memory-service requires managed persistence, a managed AWS database service — Amazon RDS for a relational store or Amazon ElastiCache for a Redis-based store — is used in place of a self-hosted database, reducing operational overhead.

5.5 CI/CD Pipeline (Jenkins)

A Jenkins pipeline is triggered automatically on every git push to the repository. The pipeline runs the automated test suite for the affected microservices, builds a new Docker image for each changed service, pushes the resulting images to the corresponding ECR repositories, and then applies the updated Kubernetes manifests to the cluster using `kubectl apply` or a Helm upgrade, thereby redeploying the changed services automatically. This establishes continuous delivery, in which a validated code commit can reach a live, running deployment without manual intervention.

5.6 Monitoring & Observability (Prometheus + Grafana)

Every microservice exposes a `/metrics` endpoint in Prometheus's exposition format, reporting request counts, request latency, and error rates. A Prometheus server scrapes these endpoints from all running services at a fixed interval and stores the resulting time-series data. Grafana is connected to Prometheus as a data source and used to build dashboards that visualize per-service health as well as cross-service call patterns — for example, the latency contributed by weather-service or memory-service to an end-to-end agent-service request — giving the team continuous visibility into the distributed system's behaviour once it is running in production.

6. DevOps Workflow

The end-to-end DevOps workflow connects a developer's code change to a live, observed production system through the following continuous sequence: a Terraform-provisioned AWS foundation exists ahead of time; a developer pushes code, which triggers Jenkins; Jenkins builds, pushes, and deploys new images to the existing Kubernetes cluster; Kubernetes runs and manages the resulting pods; and Prometheus and Grafana observe those running pods, closing the loop back to the development team.

Diagram 2: DevOps Workflow Diagram (Commit to Production, with Internal Tool Stages)

The diagram below expands the pipeline into eight sequential stages, and within each stage box the internal, tool-specific sub-steps are shown explicitly — for example, exactly what happens inside Docker during the build stage, inside Terraform during the infrastructure stage, and inside Kubernetes during deployment — rather than treating each tool as a single opaque step.

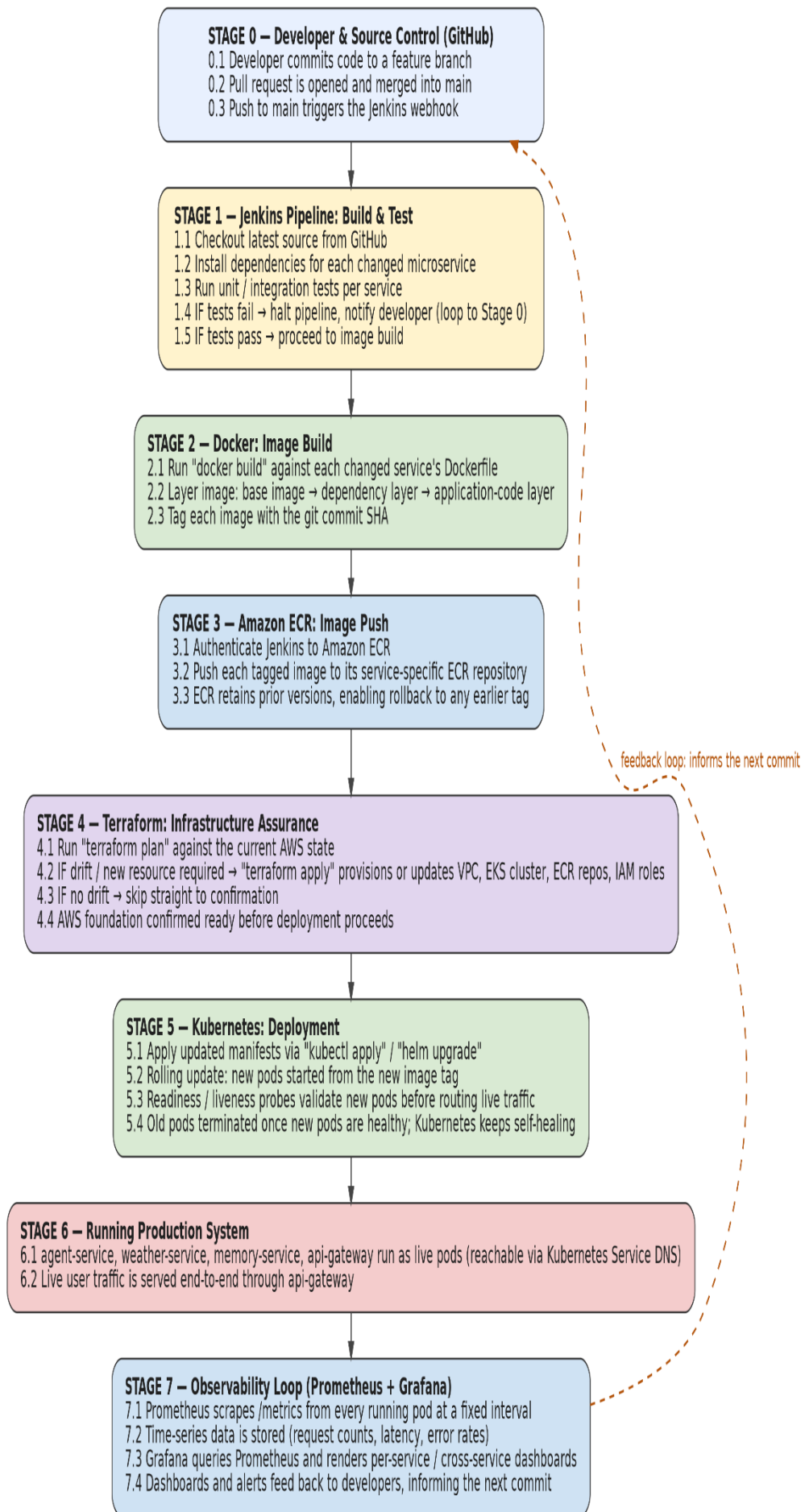


Figure 2. End-to-end DevOps workflow, from developer commit through Jenkins, Docker, ECR, Terraform, and Kubernetes to production, closed by the Prometheus/Grafana observability loop.

7. Expected Outcomes / Deliverables

- Four independently containerized microservices (agent-service, weather-service, memory-service, api-gateway), each with its own Dockerfile and image.
- A complete set of AWS infrastructure provisioned and version-controlled entirely through Terraform configuration files.
- A running Kubernetes deployment on AWS, comprising Deployments, Services, ConfigMaps, and Secrets for all four microservices.
- A working Jenkins CI/CD pipeline that builds, tests, pushes, and deploys the system automatically on every code push.
- Live Prometheus metrics collection and Grafana dashboards providing real-time observability of the deployed system.
- A finalized architecture diagram and DevOps workflow diagram documenting the system as built.

8. Tools and Technologies Summary

Category	Tool	Purpose
Containerization	Docker	Package each microservice with its dependencies into a portable, isolated image
Container Orchestration	Kubernetes	Deploy, scale, self-heal, and network the containerized microservices
Infrastructure as Code	Terraform	Declaratively provision AWS infrastructure (VPC, EKS, ECR, IAM)
Cloud Platform	AWS (EKS, ECR, RDS/ElastiCache)	Host the Kubernetes cluster, store container images, provide managed persistence
CI/CD	Jenkins	Automate build, test, image push, and deployment on every code change
Version Control	Git / GitHub	Source control and the trigger point for the CI/CD pipeline
Monitoring	Prometheus	Scrape and store time-series metrics from every microservice
Visualization	Grafana	Build dashboards for per-service and cross-service observability
Persistent State Store	Redis / PostgreSQL	Durable storage of session and trip state for memory-service
LLM Provider	Groq (OpenAI-compatible API)	Supplies the language-model reasoning consumed by agent-service

9. Timeline / Tentative Plan

- Phase 1 — Microservices Refactor: Decompose the existing monolithic agent into agent-service, weather-service, memory-service, and api-gateway, with REST interfaces defined between them.
- Phase 2 — Containerization: Write and validate a Dockerfile for each microservice; build and run the full system locally via Docker.
- Phase 3 — Kubernetes + Terraform Provisioning: Author Terraform configuration for the AWS foundation; author Kubernetes manifests; deploy the containerized services to a live EKS/Kubernetes cluster.
- Phase 4 — CI/CD Pipeline: Configure the Jenkins pipeline to build, test, push to ECR, and deploy to Kubernetes on every git push.
- Phase 5 — Monitoring Setup: Instrument each microservice with a /metrics endpoint; deploy Prometheus and Grafana; build observability dashboards.
- Phase 6 — Testing & Documentation: End-to-end system testing, finalization of both diagrams, and preparation of the project report.

10. Conclusion

The CloudPack project takes an existing, functionally complete agentic AI application and applies a complete, production-representative DevOps toolchain around it: microservice decomposition, Docker containerization, Kubernetes orchestration, Terraform-based Infrastructure as Code, AWS cloud deployment, Jenkins-driven CI/CD, and Prometheus/Grafana observability. The AI logic itself is treated as a fixed input; the project's contribution and evaluative focus lie entirely in the design and implementation of the surrounding DevOps system. On completion, the project is expected to demonstrate a fully automated path from a developer's code commit to a scalable, self-healing, observable, cloud-hosted deployment.